Sprint 2 AI Log

1. User Story 17 — Understanding the Application Notes Feature

Purpose of AI Use:
To understand the requirements of User Story 17 and determine how to implement private application notes in CareerConnect.

Prompt:
Help me understand User Story 17 — Application Notes for our CareerConnect project. Explain what needs to be implemented, which files need to be modified, and how the frontend and backend should communicate. Guide me through the implementation step by step.

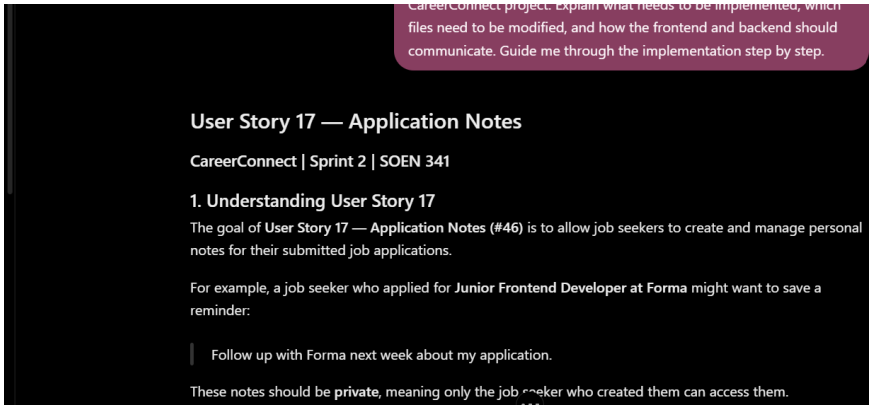
AI Suggested Content:
The AI explained that application notes needed to be associated with a specific job application and user. It suggested using a database table to store notes, backend API endpoints to create, retrieve, update, and delete notes, and a frontend interface allowing job seekers to manage their private notes.

Validation:
I reviewed the suggested implementation against the existing CareerConnect project structure and User Story 17 requirements.

Decision:
Used as guidance and adapted to the existing project.

Reflection:
The AI helped me understand how the frontend, backend, and database work together. It made the implementation process easier to follow.

Screenshot:



2. User Story 17 — Implementing the Application Notes Backend

Purpose of AI Use:
To help implement the backend functionality required to store and manage private application notes.

Prompt:
Help me implement the backend for User Story 17 in CareerConnect. I need to create, retrieve, edit, and delete private notes associated with job applications. Explain how to update schema.sql, database.js, and server.js while ensuring that users can only access their own notes.

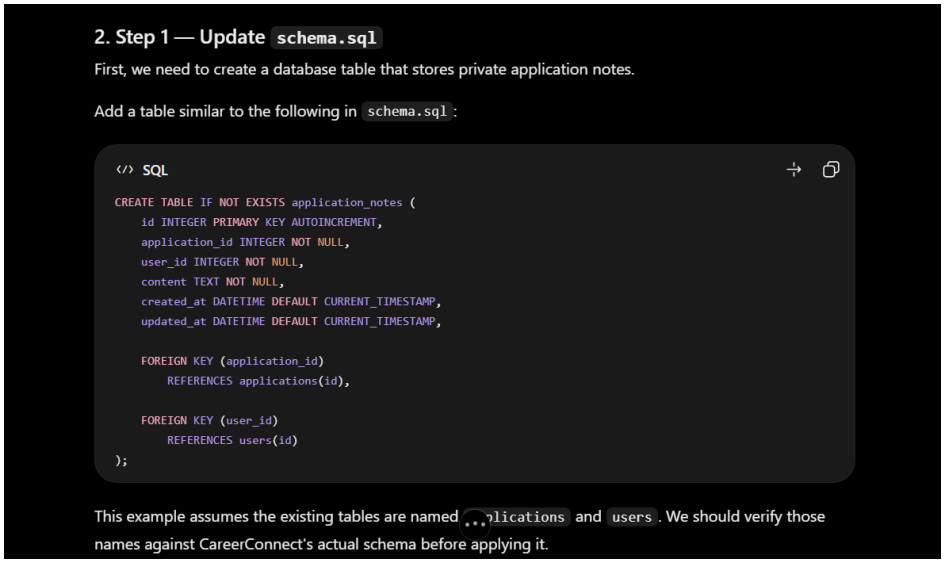
AI Suggested Content:
The AI suggested creating an application_notes database table containing identifiers for the note, application, and user. It also recommended database functions and API endpoints supporting GET, POST, PUT, and DELETE requests, with ownership checks to prevent unauthorized access.

Validation:
I reviewed the backend implementation and verified that the automated tests for adding, editing, deleting, and protecting application notes passed successfully.

Decision:
Modified before use.

Reflection:
The AI helped me understand how to structure backend operations and why user ownership checks are important when storing private information.

Screenshot:



3. User Story 17 — Implementing the Application Notes Frontend

Purpose of AI Use:
To create a user interface allowing job seekers to add, view, edit, and delete private application notes.

Prompt:
Help me connect the Application Notes backend to the CareerConnect frontend. I want a My private notes button on each application, a window displaying saved notes, and options to add, edit, and delete notes. Guide me through modifying app.js step by step.

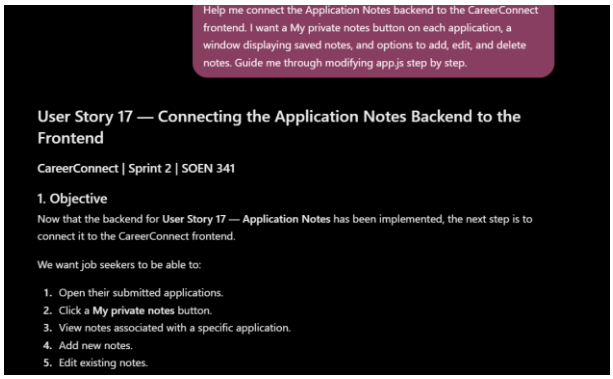
AI Suggested Content:
The AI suggested adding a My private notes button to each application card, creating modal windows for viewing and editing notes, and connecting the interface to the backend API. It also explained how to modify the requestData function to support PUT and DELETE requests.

Validation:
I implemented the frontend changes in app.js and manually tested the interface. The My private notes button appeared correctly, and the application notes window opened successfully.

Decision:
Modified before use.

Reflection:
The AI helped me understand how frontend event handlers and API requests work together. The step-by-step guidance made it easier to implement the feature without losing track of the changes.

Screenshot:



4. User Story 17 — Testing and Merging the Implementation

Purpose of AI Use:
To verify that the Application Notes feature worked correctly and integrate the completed frontend changes into the GitHub repository.

Prompt:
I finished implementing the Application Notes feature. Help me test adding, editing, and deleting notes, then guide me through creating a GitHub branch, committing my changes, opening a pull request, and merging it into main.

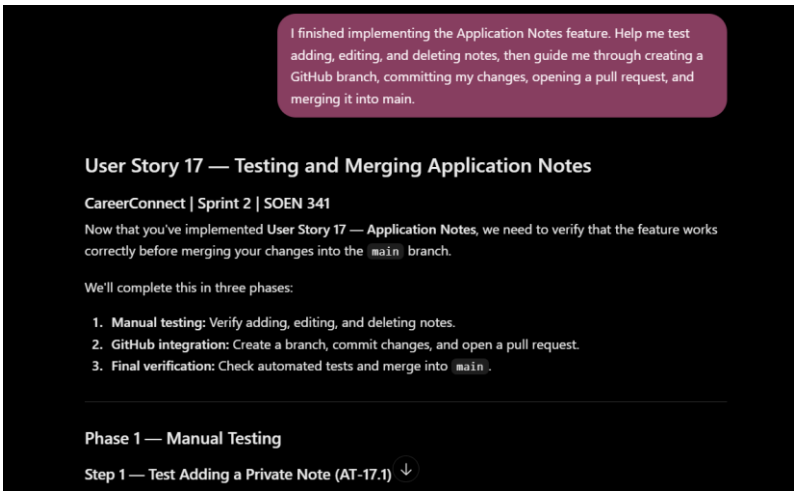
AI Suggested Content:
The AI recommended manually testing the note creation, editing, and deletion functions. It also provided step-by-step instructions for creating a frontend branch, committing app.js, opening a pull request, reviewing automated checks, and merging the changes.

Validation:
I successfully tested adding, editing, and deleting notes. I created the feature/application-notes-frontend branch, opened Pull Request #87, verified that the checks passed, and merged the changes into main.

Decision:
Used as guidance and verified each step manually.

Reflection:
The AI helped me understand the GitHub workflow and how to integrate my changes safely. I also learned the importance of testing a feature before merging it into the main branch.

Screenshot:



5. AT-17.1 — Add and Save Application Notes

Purpose of AI Use:
To prepare and refine the acceptance test for adding and saving private application notes.

Prompt:
Help me refine AT-17.1 for User Story 17 using the manual testing results and screenshots. Keep the same acceptance-test format and update the test steps, expected results, actual results, test status, and evidence.

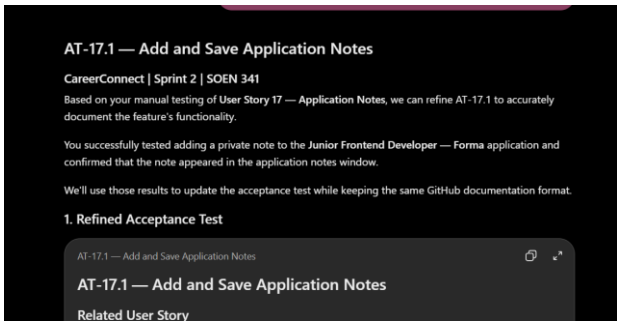
AI Suggested Content:
The AI recommended testing the My private notes button, entering a personal note, saving it, and verifying that it appeared under the correct job application. It also suggested documenting the saved note and associated screenshot evidence.

Validation:
I tested the feature using the Junior Frontend Developer application at Forma. I successfully added a note and verified that it appeared in the Private application notes window.

Decision:
Modified before use.

Reflection:
The AI helped me organize the acceptance test and clearly explain how the feature was verified through manual testing.

Screenshot:



6. AT-17.2 — Edit and Delete Application Notes

Purpose of AI Use:
To verify that users can modify and remove their saved application notes.

Prompt:
Help me update AT-17.2 for User Story 17. I successfully edited and deleted a private application note. Use my testing results to refine the test steps, actual results, evidence, and screenshot descriptions.

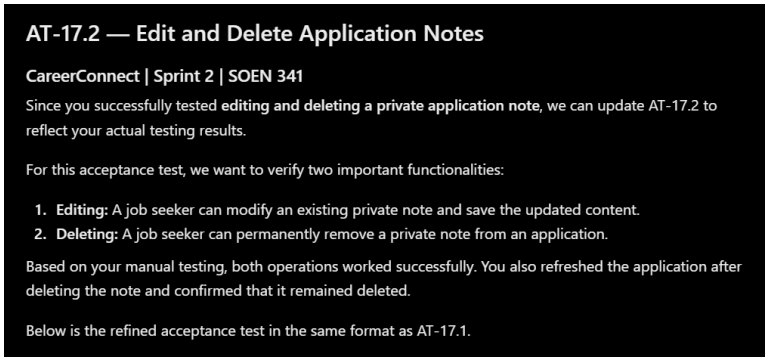
AI Suggested Content:
The AI recommended documenting the editing process, verifying that updated note content appeared correctly, and confirming that deleted notes disappeared. It also suggested refreshing the page to verify that deleted notes did not reappear.

Validation:
I manually edited a saved note, confirmed that the updated content appeared, deleted the note, and verified that it remained deleted after refreshing the page.

Decision:
Modified before use.

Reflection:
The AI helped me create a more complete acceptance test by including both editing and deletion scenarios and identifying the evidence needed to support the results.

Screenshot:



7. AT-17.3 — Application Notes Privacy

Purpose of AI Use:
To verify that application notes are private and cannot be viewed by another job seeker.

Prompt:
Help me test AT-17.3 — Application Notes Privacy. I want to use two different job seeker accounts to verify that a note created by one user cannot be viewed by another user. Guide me through the test and explain what screenshots I need.

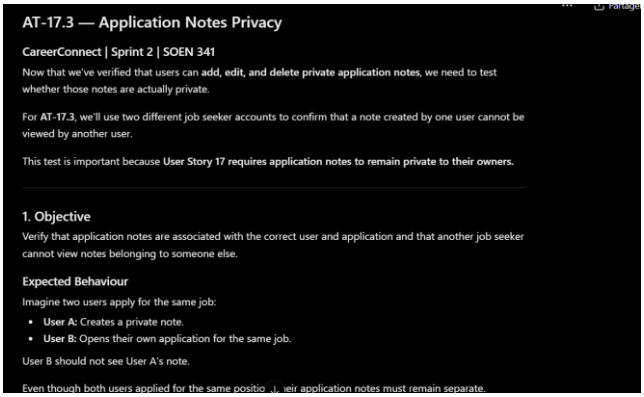
AI Suggested Content:
The AI recommended creating a private note using the first account, logging out, and accessing the same job posting through a second account. It suggested comparing the notes displayed for each account and documenting the results with screenshots.

Validation:
I created the note "PRIVATE TEST — Follow up with Forma on November 2" using the first account. When I logged into the second account, the note was not visible, and the interface displayed "No private notes yet."

Decision:
Used as suggested.

Reflection:
The AI helped me understand how to test privacy using multiple accounts. This test demonstrated why user-specific access controls are important.

Screenshot:



8. AT-16.1 — Save a Job to Favourites

Purpose of AI Use:
To verify that job seekers can save job postings to their favourites.

Prompt:
Help me test AT-16.1 — Save a Job to Favourites for CareerConnect. Explain the steps I should follow and how to document the results using screenshots.

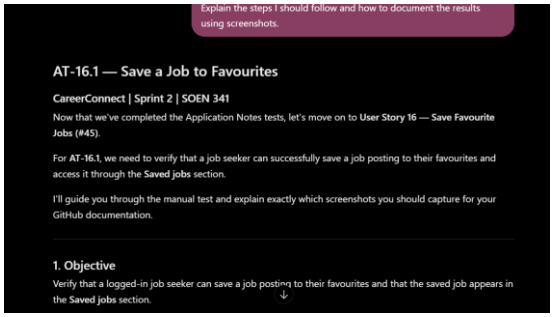
AI Suggested Content:
The AI recommended selecting a job posting, clicking its bookmark icon, navigating to Saved jobs, and verifying that the selected job appeared with the correct information.

Validation:
I saved the Junior Frontend Developer position at Forma and confirmed that it appeared in the Saved Jobs section. The page displayed "1 opportunity on your shortlist."

Decision:
Modified before use.

Reflection:
The AI helped me identify the steps required to verify the save-job functionality and organize the evidence for my acceptance test.

Screenshot:



9. AT-16.2 — View Saved Jobs

Purpose of AI Use:
To verify that multiple saved job postings appear together in the Saved Jobs section.

Prompt:
Help me test AT-16.2 — View Saved Jobs. I want to verify that two saved jobs appear together and display the correct job information. Explain what I should check and how to document the results.

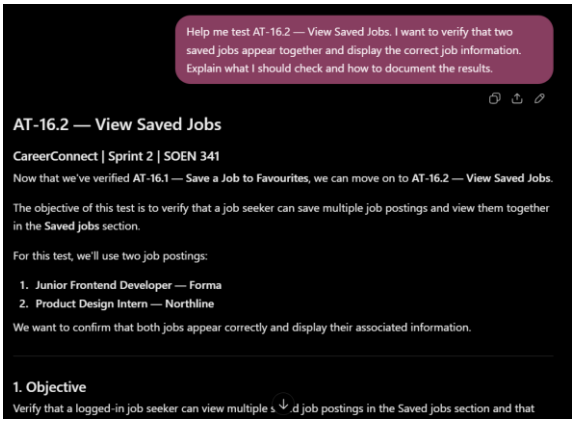
AI Suggested Content:
The AI recommended saving two different job postings, opening the Saved Jobs section, and checking the company names, job titles, locations, employment types, salary ranges, and skills.

Validation:
I saved Junior Frontend Developer at Forma and Product Design Intern at Northline. Both appeared together in the Saved Jobs section with their job information and upcoming application deadlines.

Decision:
Used as guidance.

Reflection:
The AI helped me understand how to test a feature using multiple records instead of checking only one saved job.

Screenshot:



10. AT-16.3 — Remove a Saved Job

Purpose of AI Use:
To verify that users can remove a saved job without affecting other saved postings.

Prompt:
Help me refine AT-16.3 — Remove a Saved Job. I removed one job from my favourites and confirmed that another saved job remained. Update the acceptance-test documentation using my results.

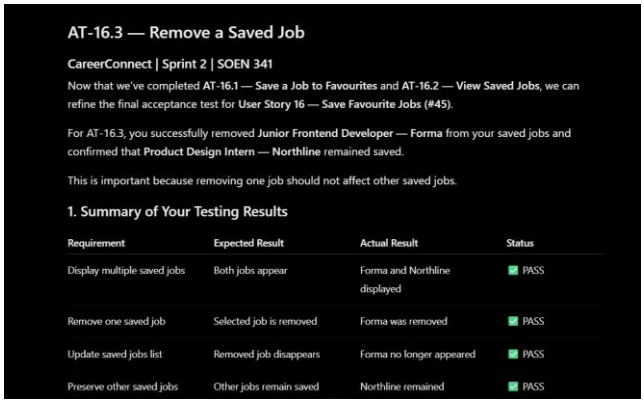
AI Suggested Content:
The AI recommended removing one saved job, verifying that a confirmation message appeared, checking that the removed job disappeared, and confirming that the remaining saved jobs were unaffected.

Validation:
I removed Junior Frontend Developer at Forma. CareerConnect displayed "Job removed from saved," and the Saved Jobs section continued displaying Product Design Intern at Northline.

Decision:
Modified before use.

Reflection:
The AI helped me document the removal process and understand the importance of verifying that an action does not unintentionally affect other saved records.

Screenshot:



AT-16.3 — Remove a Saved Job

CareerConnect | Sprint 2 | SOEN 341

Now that we've completed **AT-16.1 — Save a Job to Favourites** and **AT-16.2 — View Saved Jobs**, we can refine the final acceptance test for **User Story 16 — Save Favourite Jobs (#45)**.

For AT-16.3, you successfully removed **Junior Frontend Developer — Forma** from your saved jobs and confirmed that **Product Design Intern — Northline** remained saved.

This is important because removing one job should not affect other saved jobs.

1. Summary of Your Testing Results

Requirement	Expected Result	Actual Result	Status
Display multiple saved jobs	Both jobs appear	Forma and Northline displayed	✅ PASS
Remove one saved job	Selected job is removed	Forma was removed	✅ PASS
Update saved jobs list	Removed job disappears	Forma no longer appeared	✅ PASS
Preserve other saved jobs	Other jobs remain saved	Northline remained	✅ PASS

11. AT-18.1 — Application Progress Dashboard

Purpose of AI Use:
To verify that the application dashboard displays accurate application status counts.

Prompt:
Help me test AT-18.1 — Application Progress Dashboard. I want to verify that the dashboard displays the correct application counts and updates when I submit another application. Guide me through the testing process and help me identify any missing requirements.

AI Suggested Content:
The AI recommended comparing the dashboard counts with the applications listed in Application History, submitting an additional application, and checking whether the Applied count increased. It also suggested testing the status filter and documenting any missing dashboard elements.

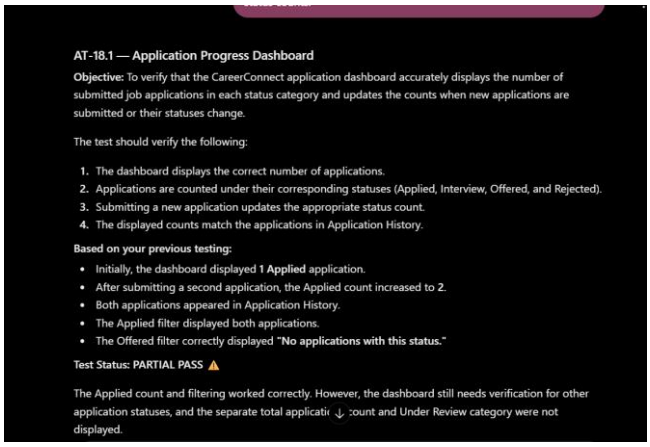
Validation:
I initially had one submitted application, and the dashboard displayed 1 Applied. After submitting a second application, the count increased to 2. Both applications appeared in Application History. I also verified that the Applied filter displayed both applications and that the Offered filter displayed an appropriate empty-state message.

The AI helped identify that the dashboard did not display a separate Total Applications counter or an Under Review category.

Decision:
Modified before use.

Reflection:
The AI helped me compare the implemented dashboard with the original acceptance criteria. I learned that an acceptance test should document missing requirements rather than automatically marking a partially implemented feature as fully passed.

Screenshot:



12. Acceptance-Test Documentation and Screenshot Evidence

Purpose of AI Use:
To improve the consistency and completeness of my Sprint 2 acceptance-test documentation.

Prompt:
Help me organize my CareerConnect acceptance tests using a consistent GitHub format. Each test should include the related user story, related task, objective, preconditions, test steps, expected results, actual results, test status, evidence, and screenshots. Make sure the results accurately reflect what was tested.

AI Suggested Content:
The AI suggested a consistent structure for acceptance tests and recommended using screenshots to demonstrate the actual functionality. It also emphasized distinguishing between completed tests, partially completed tests, and requirements that had not yet been verified.

Validation:
I reviewed the updated acceptance tests against my manual testing results and screenshots for User Stories 16, 17, and 18.

Decision:
Modified before use.

Reflection:
The AI helped me make my acceptance-test documentation clearer and more consistent. It also helped me identify missing test coverage before the Sprint 2 deadline.

Screenshot:



CareerConnect — Organizing Sprint 2 Acceptance Tests

SOEN 341 | GitHub Documentation

To keep our Sprint 2 acceptance tests consistent, I recommend using **one standardized format for every test.**

This will make the documentation easier for teammates to review and help ensure that the results accurately reflect what was tested.

1. Standard Acceptance-Test Format

Each GitHub acceptance test should contain the following sections:

Section	Purpose
Related User Story	Identifies the user story being tested
Related Task	Identifies the specific functionality
Objective	Explains what the test verifies

13. Reviewing and Understanding the Application Notes Code

Purpose of AI Use:
To better understand the coding process for User Story 17 — Application Notes and how the frontend, backend, and database work together.

Prompt:
Help me review the code implementation of User Story 17 — Application Notes in CareerConnect. Explain how the changes in schema.sql, database.js, server.js, and app.js work together. Focus on the CRUD functions, API requests, event handlers, and user authentication. Explain why each modification is necessary and how I can verify that the code works correctly.

AI Suggested Content:
The AI explained how to create the application_notes database table, implement CRUD functions in database.js, and connect them to API endpoints in server.js. It also explained how to modify app.js to add the My private notes button, display notes in a modal, and handle adding, editing, and deleting notes. The AI emphasized the importance of checking user ownership to ensure notes remain private.

Validation:
I compared the AI explanations with the code implemented in CareerConnect and verified the functionality through manual testing. I successfully tested adding, editing, and deleting notes, as well as checking note privacy using two different job seeker accounts.

Decision:
Used as guidance and adapted to the existing project structure.

Reflection:
The AI helped me better understand how different parts of the code communicate with each other. It also helped me understand how API requests work and why authentication and ownership checks are important when handling private user information.

Screenshot:

